🔥 Project: Building a Custom Headline Generator using Bigram & Trigram Language Models

🎯 Problem Statement

Given a dataset of real news headlines (you can take the **AG News dataset's titles** or scrape headlines from a site like BBC/Reuters), build a **statistical text generation system** that produces realistic-looking news headlines. The project should:

1. **Preprocess the text** using NLTK (tokenization, lowercasing, stopword removal optional).
2. **Build n-gram language models** (bigram & trigram) with NLTK's ConditionalFreqDist.
3. **Implement text generation** by probabilistically sampling the next word given a context (bigram/trigram).
4. **Compare outputs** between bigram and trigram models.
5. **Evaluate quality** with **perplexity** or by holding out test headlines.
6. (Optional Advanced) Add **smoothing techniques** (like Laplace) for unseen n-grams.

🛠 Steps for Students

1. **Dataset Preparation**
    - Use AG News dataset headlines (from Hugging Face or CSV).
    - Clean and tokenize with NLTK.
2. **Exploration**
    - Frequency distribution of words.
    - Visualize top bigrams/trigrams.
3. **Model Building**
    - Build a **Bigram model** using ConditionalFreqDist.
    - Extend to **Trigram model**.
4. **Text Generation Function**
    - Implement a function that starts with a random word and generates N tokens using the model.

5. **Comparison**
    - Generate 5 sample headlines with the bigram model.
    - Generate 5 sample headlines with the trigram model.
    - Discuss differences.
6. **Evaluation**
    - Compute perplexity on a test split.
    - Discuss limitations of n-gram models.

🚀 **Learning Outcomes**

- Learn **tokenization, n-grams, language modeling** with NLTK.
- Understand the difference between **bigram vs trigram** generation quality.
- Learn about **data sparsity** & **need for smoothing/transformer models**.
- Realize why modern LLMs outperform n-gram methods.